The Myth of the "Living" Document: Are OpenSpec's Living Specifications Ever Revised?

Tran Vo Thanh Nghiep Quan
Dep. of Computer and Software Engineering, Polytechnique Montréal, Montréal, Canada

***Abstract*—In spec-driven development (SDD), developers describe what to build in a Markdown specification and an AI coding agent implements it. OpenSpec, the SDD tool with the most spec files in SpecMine (the MSR 2027 Mining Challenge dataset), keeps one *living* spec per capability and calls it the source of truth. We ask whether living specs are actually maintained. In a preliminary analysis of 72,971 living specs, 62.7% were never revised after creation, and 19.6% (14,294 specs) are *fossils*: they still hold a TBD placeholder in their Purpose section, usually the one OpenSpec's archive step writes, and nobody ever edited them. We propose three research questions on revision behaviour, fossil prevalence and characterization, and cross-tool comparison.**

I. Problem Statement

SDD tools such as OpenSpec [1] and GitHub Spec Kit [2] make the specification the input of AI coding agents [3]. In OpenSpec, a developer proposes a change (`changes/<name>/` with a proposal, delta specs, a design and tasks), the agent implements it, and *archiving* moves the change to `changes/archive/` and merges its delta specs into `openspec/specs/`. OpenSpec calls these specs the source of truth for how the system currently behaves [1], and agents read them before every later change. A spec that stops changing while the code moves on is wrong input for the next agent.

OpenSpec gives us an unusual marker of unfinished documentation. Before v1.7.0, archiving a change that introduced a new capability always wrote "`TBD - created by archiving change <name>. Update Purpose after archive.`" into the new spec's Purpose section [4]. The fix (PR #1431) shipped on July 29, 2026, after SpecMine's crawl ended (July 11), so any spec in the snapshot could have received the placeholder. A spec that still holds it and was never revised was written once and never finished; it is also stale only if its code moved on (RQ2).

Outdated documentation is a long-standing problem [5], [6], [7]. The SpecMine authors list spec lifecycle, including "placeholders never filled," as an open question [8]. Their practitioners report *spec decay*, which commit counts alone cannot show [8]; we add a tool-written marker.

Link with the course. Living specs are configuration for AI agents, kept as code; their staleness mirrors configuration drift. OpenSpec has since added a validation check for the placeholder [9], a gate a CI pipeline can enforce; we measure how often such a gate would fire on existing specs.

II. Research Questions

All analyses start from *commits per spec file*. A *revision* is a commit that touches the file after its creation commit. We separate *archive merges* (commits that also create a file under `changes/archive/`) from other edits. A *fossil* is a living spec whose snapshot content holds an unfilled placeholder and that has zero revisions.

RQ1: Do living specs get revised? *Motivation:* if living specs are the source of truth, each archived change that touches a capability should revise its spec. *Approach:* revisions per living spec, split into archive merges and other edits; Kaplan–Meier curves of time to first revision, censored at the snapshot; 95% CIs from a bootstrap over repositories, as fossils cluster (Fig. 1); χ^2 tests with Cramér's V.

RQ2: How prevalent are fossil living specs, and what kinds of specs become fossils? *Motivation:* the naive "never revised" rate mixes finished specs with forgotten ones; the placeholder helps tell them apart. *Approach:* we report the shares of fossils, of specs whose placeholder survived a revision and of specs without one, among all living specs and among specs born with the placeholder (34%–67% from snapshot data), against the naive measure and a no-placeholder control group. We then ask what fossils look like, as the instructor suggested. Per repository, we sample at most one fossil and, for contrast, one *fixed* spec (first version with the placeholder, snapshot without; found with `git log -p` on cloned repositories), with a fixed seed. Each gets automatic features (repository activity, bulk creation, size, later code changes, which separate stale fossils from unfinished ones, and who filled a fixed spec: a human, an agent or a bulk update) and a manual category: both students open-code 30 specs into a codebook and double-code 100 (Cohen's κ). An LLM (temperature 0, tuned on the pilot only) labels the other ≈375 fossils (95% confidence, ±5%) if it reaches $\kappa \geq 0.$ on held-out human labels.

RQ3: Is write-once the same across tools? *Motivation:* only OpenSpec encodes lifecycle roles [8]; Spec Kit's per-feature specs resemble OpenSpec's change specs, not its living specs, and archived specs restart their history at the archive move (96.7% unrevised). *Approach:* compare like with like across tools, against the pooled comparison.

Robustness. File age: 22% of living specs are under 30 days old, so we re-run every result without specs younger than 30, 60 and 90 days (for specs at least 90 days old, 58.0% were never revised and 16.8% are fossils). Repository dominance: per-repository caps and leave-largest-out. Inactive repositories separate forgotten from finished specs.

III. Dataset

We use SpecMine v1.0 (July 2026 snapshot) [10], [8]: 470,795 spec files from 73,030 GitHub repositories. OpenSpec

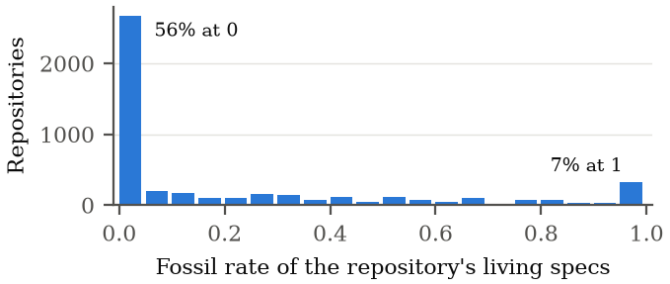


Figure 1. Fossil rate per repository (≥3 living specs; 4,618 repositories).

Table I
PRELIMINARY RESULTS FOR OPENSPEC LIVING SPECS (WILSON CIS)

Quantity	Value	95% CI
Living / delta / archived specs	72,971	/ 41,582 / 156,372
Living specs never revised	62.7%	[62.4, 63.1]%
Unfilled placeholder	29.2%	[28.9, 29.6]%
and never revised (fossil)	19.6%	[19.3, 19.9]%
and revised at least once	9.6%	
Repos with fossils (largest share: 0.94%)	2,488	

accounts for 274,955 files (58.4%) in 8,926 repositories. We query the Parquet tables with DuckDB: `spec_files` (tool, lifecycle role), `spec_file_commits` (per-file history) and `spec_content_features` (the placeholder flag), plus snapshot texts (`specs.jsonl.gz`, Zenodo only). Our cohort keeps the 468,163 specs with both a fetched commit history and fetched content (735,667 commits); the 2,632 excluded files lack history (1,473) or content (2,488); 1,329 of them lack both. Table I gives our preliminary numbers. Fig. 1 shows that fossils cluster by repository: 70% of repositories with at least 3 living specs have a fossil rate at or below 10% or at or above 90%, so we sample fossils per repository.

Data caveats we already checked. (1) SpecMine histories do not follow renames. In 29 of 30 sampled specs, SpecMine's commit count equals a path-only `git log`, and 1 of the 15 living specs among them was renamed. Revision counts are therefore lower bounds. (2) The placeholder flag describes the snapshot only; it catches all 19,651 specs with the archive text (recall 100.0%), and 92.2% of flagged specs hold that text, 98.7% some TBD Purpose. (3) SpecMine has no code commits outside its pull-request layer, so code activity and fixed specs require cloning repositories.

IV. RELATED WORK

SDD and SpecMine. Agarwal et al. [8] release SpecMine and study SDD through interviews and 266 repositories; they report that living specs decay but do not measure it. We do, with per-file history over all 72,971 living specs.

Outdated documentation. Documentation is often out of date [5], up-to-dateness is part of documentation-issue taxonomies [6], and outdated code references can be detected in READMEs and wikis [7]; we study specs the tool labels current.

Self-admitted technical debt. Potdar and Shihab [11] and Bavota and Russo [12] study debt that developers admit in code comments and whether it is removed. In our setting the tool writes the debt marker, and we ask whether anyone pays it.

V. TIME PLANNING OF PROJECT

Fig. 2 shows the plan.

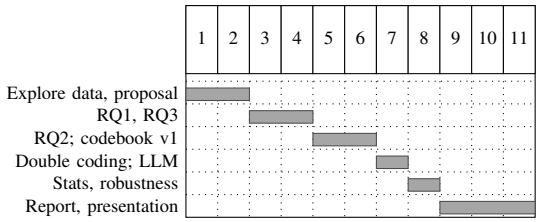


Figure 2. Schedule by week (week 1 starts Sep. 28, 2026; the report is due Dec. 18).

REFERENCES

[1] Fission-AI, "OpenSpec: Repository README and Concepts documentation," 2026, accessed: Oct. 18, 2026. [Online]. Available: https://github.com/Fission-AI/OpenSpec
[2] GitHub, "Spec Kit," 2025, accessed: Oct. 18, 2026. [Online]. Available: https://github.com/github/spec-kit
[3] B. Böckeler, "Understanding spec-driven-development: Kiro, spec-kit, and Tessl," martinfowler.com, Oct. 2025, accessed: Oct. 18, 2026. [Online]. Available: https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
[4] Fission-AI/OpenSpec, "fix(archive): keep the delta spec's Purpose in a new main spec," GitHub pull request #1431, Jul. 2026, released in v1.7.0. [Online]. Available: https://github.com/Fission-AI/OpenSpec/pull/1431
[5] T. C. Lethbridge, J. Singer, and A. Forward, "How software engineers use documentation: The state of the practice," *IEEE Software*, vol. 20, no. 6, pp. 35–39, 2003.
[6] E. Aghajani, C. Nagy, O. L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, and M. Lanza, "Software documentation issues unveiled," in *Proc. IEEE/ACM 41st Int. Conf. on Software Engineering (ICSE)*, 2019, pp. 1199–1210.
[7] W. S. Tan, M. Wagner, and C. Treude, "Detecting outdated code element references in software repository documentation," *Empirical Software Engineering*, vol. 29, no. 1, Jan. 2024, art. no. 5.
[8] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux, "Spec-driven development as prescribed and as practiced," arXiv:2608.25202v4 [cs.SE], Oct. 2026.
[9] Fission-AI/OpenSpec, "fix(validate): report a Purpose left as the archive placeholder," GitHub pull request #1671, Aug. 2026, released in v1.12.0. [Online]. Available: https://github.com/Fission-AI/OpenSpec/pull/1671
[10] S. Agarwal and B. Vasilescu, "SpecMine: A large-scale corpus of spec-driven development artifacts (v1.0)," Zenodo, doi:10.5281/zenodo.22102780, Aug. 2026.
[11] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2014, pp. 91–100.
[12] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *Proc. 13th Int. Conf. on Mining Software Repositories (MSR)*, 2016, pp. 315–326.

Index des commentaires

1.1 Promising study. Suggestions if you want to submit this to MSR mining challenge later:
RQ1 -> portion of living/fossil specs
RQ2 -> characterize living specs: spec category, first revision, etc.
RQ3 -> characterize fossil specs
RQ4 -> how do living and fossil specs differ from each other
- Categories for specifications may be found in previous literature

1.2 Briefly explain why

1.3 Good list of crucial definitions! Define "living specs" too

1.4 Maybe "How many/portion living specs get revised"?

1.5 I feel like this part can be answered in RQ1? In RQ1, you can check the portion of specs that got revised and became fossil, etc.

1.6 What is a spec? Define it briefly

1.7 Suggestion: you may check related works for predefined categories. In that case, you don't have to manually come up with category labels

1.8 Good introduction. In the final report, I suggest adding an example (e.g., a figure) of a spec in this dataset, and a figure demonstrating how the agents interact with each other

1.9 Good!

1.10 What is "write-once"?

1.11 What does it mean?